

Hum-to-Search

AI MELODY-BASED SONG RECOGNITION SYSTEM

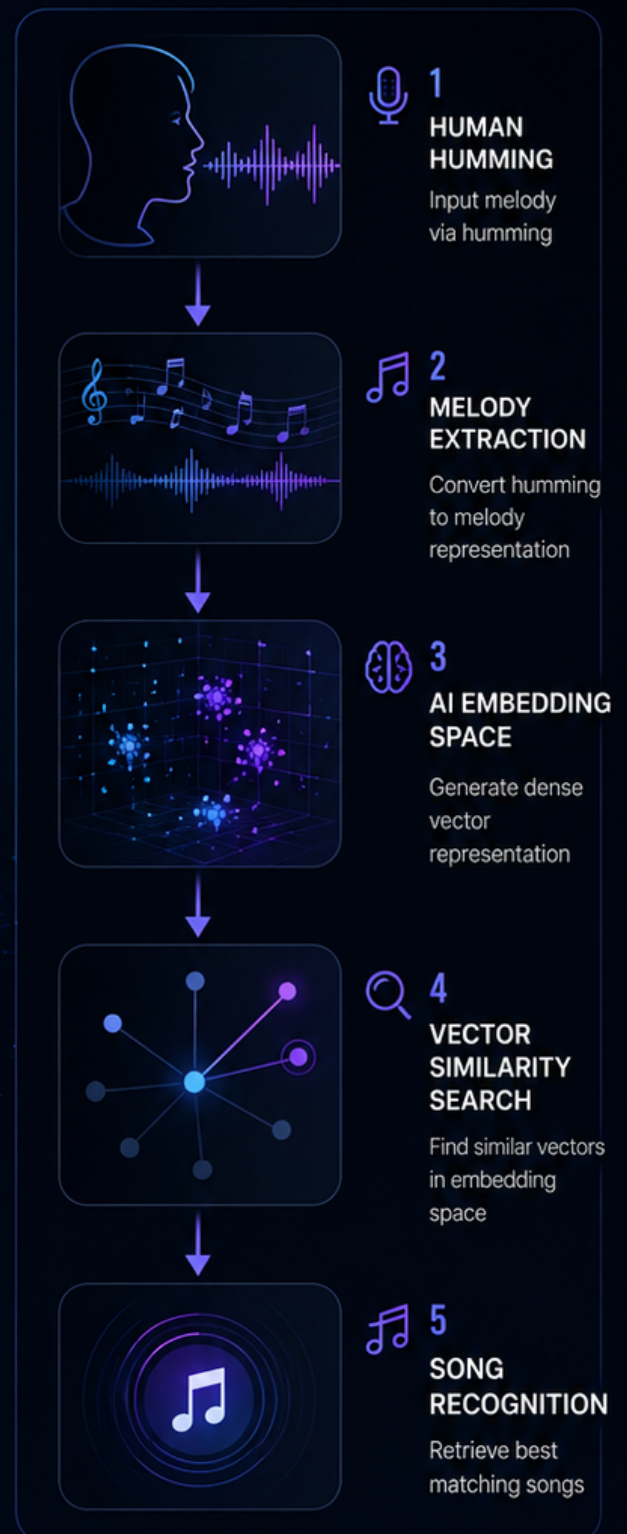
An AI-powered system for retrieving songs using humming-based melody understanding



Author
Umair Ahmed

Category

Artificial Intelligence | Audio Processing
Machine Learning | Vector Search



POWERED BY



Python



Demucs



BasicPitch



Qdrant



PyTorch



TensorFlow



Streamlit

Table of Contents

1. Executive Summary

2. Problem Statement

3. Project Objectives

4. System Architecture

5. Technical Implementation

6. Technology Stack

7. Engineering Challenges and Mitigations

8. Evaluation

9. Comparison with Existing Approaches

10. Future Roadmap

11. Engineering Learnings

12. Conclusion

Hum-to-Search: AI Melody-Based Song Recognition System

Technical Project Report

Repository: github.com/umair2213/Hum-to-search

1. Executive Summary

Hum-to-Search is a complete, offline AI system for **query-by-humming (QBH)** song recognition — identifying songs from a hummed melody or uploaded audio clip using only the *melodic contour* of the signal, with deliberate invariance to musical key, vocal range, and tempo.

The system chains six AI/ML stages into a unified pipeline: **blind source separation** (Demucs `htdemucs`) to isolate vocal stems from polyphonic mixes, **neural pitch tracking** (BasicPitch, Spotify ICASSP 2022) to extract discrete note events, **domain-specific signal cleaning** to suppress extraction artifacts, **transposition-invariant interval encoding** via first-order differencing, **deterministic feature embedding** into a 768-dimensional L2-normalized vector, and **approximate nearest neighbor (ANN) retrieval** via Qdrant's HNSW index.

The system operates entirely offline on local hardware with no external API dependencies. Indexing is parallelized using a `ThreadPoolExecutor`, leveraging GIL-releasing inference in PyTorch (Demucs) and TensorFlow (BasicPitch). The Streamlit-based UI supports both live microphone recording and audio file upload. All deterministic pipeline stages are covered by a `pytest` unit test suite.

2. Problem Statement

2.1 The Limits of Conventional Music Search

Approach	Query Modality	Core Limitation
Lyrics search	Typed text	Requires lexical recall; fails for instrumentals or when the user knows the tune but not the words
Audio fingerprinting (Shazam-style)	Exact audio playback	Computes a spectral hash of the original recording; a hum produces an entirely different spectral signature and cannot be matched
Collaborative filtering	Listening history	Solves music <i>recommendation</i> , not <i>recognition</i> ; requires user history

2.2 Why Query-by-Humming Is Hard

QBH is a fundamentally harder subproblem within music information retrieval (MIR) because:

- **Signal degradation.** A human hum is monophonic, imprecisely pitched, tempo-inconsistent, and recorded in uncontrolled acoustic environments — far noisier than any studio recording.
- **Absence of lexical and timbral anchors.** Unlike text search or fingerprinting, no lyrics, spectral signatures, or identifying timbres survive in a hum. Only the relative pitch contour remains.
- **Transposition variance.** Users hum in their natural vocal range, not in the song's original key. Any key-dependent representation will fail to match despite melodic identity.

The core insight of this project is that **relative melodic contour** — the sequence of pitch directions and magnitudes — is the one feature class that is both preserved in humming and sufficient for identification. The entire feature engineering pipeline is designed to extract, denoise, and represent this signal robustly.

3. Project Objectives

- Accept a live microphone recording or uploaded audio file and return a ranked list of matching songs.
- Isolate vocal stems from full polyphonic mixes to produce indexed representations comparable to monophonic hum input.

- Convert raw audio into discrete note events using a pretrained neural pitch tracker.
 - Remove known artifact classes (false pitches, octave errors, vibrato splitting) introduced by the pitch tracker.
 - Represent melody as a fixed-length vector that is invariant to uniform key transposition and normalized for tempo differences.
 - Store and retrieve embeddings in a vector database with sub-linear retrieval complexity.
 - Deliver production-grade engineering: parallelized indexing with disk-cached intermediates, a Streamlit UI with real-time feedback, and unit-test coverage for all deterministic components.
-

4. System Architecture

4.1 Asymmetric Dual-Pipeline Design

The system employs an **asymmetric dual-pipeline architecture** — a design choice driven by the fundamental difference between the indexing and query inputs.

A full song mix requires source separation before pitch tracking because the pitch tracker would otherwise detect notes from every instrument simultaneously. A hummed input is already monophonic vocal audio and requires no separation. Running Demucs at query time would add 15–30 s of latency with no representational benefit. Both pipelines converge at the melody extraction stage and follow an identical path to the final embedding.

4.2 Indexing Pipeline (Offline, Parallelized)

```
Song File (.mp3 / .wav / .flac / .ogg / .m4a)
|
▼
Demucs – htdemucs (hybrid time-frequency Transformer)
|   Blind source separation; isolates vocal stem from polyphonic mix
|   vocals.wav cached to disk → subsequent runs skip this stage
▼
BasicPitch – ICASSP 2022 CNN
|   Neural pitch tracking; frame-level onset / frame / offset maps
|   Output: [{pitch: int, start: float, end: float}, ...]
▼
Melody Filter – 5-Stage Deterministic Pipeline
|   (1) pitch range + duration gate
|   (2) overlapping duplicate suppression (non-maximum within window)
|   (3) sequential pitch-jump outlier removal (> 12 semitones)
|   (4) harmonic overtone filter (exact 12 / 24 / 36 semitone jumps)
|   (5) vibrato smoothing (merge adjacent notes within ≤ 2 semitones)
▼
Interval Normalization – First-Order Differencing
|   intervals[i] = pitch[i] - pitch[i-1]
|   Produces a transposition-invariant interval sequence
▼
Embedding – 768-dim Deterministic Feature Vector
|   [49-dim histogram | 712-dim contour | 7-dim statistics] → L2-normalized
▼
Qdrant (Local Embedded) – HNSW Cosine Index
|   Stores vector + JSON payload {title, path}; persisted to disk
```

4.3 Query Pipeline (Online, Low-Latency)

```
Hum Recording / Uploaded Audio File
|
▼
BasicPitch – ICASSP 2022 CNN      ← No Demucs; input is already monophonic
▼
Melody Filter (same 5 stages)
▼
Interval Normalization
▼
Embedding (same 768-dim construction)
▼
Qdrant – HNSW Cosine ANN Search
    Returns top-K results with cosine similarity ≥ 0.20
```

4.4 Key Architectural Decisions

Decision	Rationale
Asymmetric pipelines	Demucs skipped at query time: hum input is already monophonic; applying BSS adds latency without representational benefit
Disk-cached vocal stems	Demucs is the most compute-intensive stage (15–30 s/song); caching <code>vocals.wav</code> eliminates redundant separation on re-indexing runs
ThreadPoolExecutor parallelism	Both Demucs (PyTorch) and BasicPitch (TensorFlow) release the Python GIL during inference, enabling true multi-core throughput without multiprocessing overhead
Deterministic (non-learned) embedding	Eliminates training data requirements; fully interpretable; appropriate for a system without labeled hum/song pairs
Local embedded Qdrant	No external server process; HNSW index scales to larger collections; clean interface between storage and feature engineering

4.5 Component-to-Module Mapping

Pipeline Stage	Module	Responsibility
Vocal separation	<code>src/audio/separator.py</code>	Demucs inference + disk cache check and write
Note extraction	<code>src/audio/melody.py</code>	BasicPitch inference; returns canonicalized note dicts
Signal cleaning	<code>src/audio/melody_filter.py</code>	Five-stage deterministic artifact removal
Interval encoding	<code>src/audio/melody_normalizer.py</code>	First-order differencing of pitch sequence
Feature embedding	<code>src/embedding/melody_embedder.py</code>	Histogram + contour + statistics → L2-normalized vector
Vector storage	<code>src/database/qdrant_client.py</code>	Collection lifecycle, upsert, cosine query
Indexing orchestration	<code>src/index_songs.py</code>	Parallel pipeline over full song library
Query orchestration	<code>src/search_song.py</code> , <code>src/search/search_engine.py</code>	Query embedding generation + search dispatch
UI	<code>app.py</code>	Streamlit: record, upload, search, rebuild index

5. Technical Implementation

5.1 Vocal Separation — Blind Source Separation via Demucs

Problem. A full mix is a linear superposition of drums, bass, harmonic instruments, and vocals. Neural pitch trackers applied directly to the mix detect notes from all active sources simultaneously, producing a representation dominated by the loudest source rather than the vocal melody.

Solution. `src/audio/separator.py` invokes Demucs (`htdemucs`) to perform **blind source separation (BSS)** — decomposing the mixture into its constituent stems without prior knowledge of

the sources. The `htdemucs` model uses a **hybrid time-frequency Transformer** that processes both waveform and spectrogram representations in parallel, trained on the MUSDB18HQ dataset to isolate vocals, drums, bass, and other into separate stems.

Representational symmetry. By isolating the vocal stem before pitch tracking, the extracted note sequence reflects the sung melody — the same signal modality produced by a human hum. This symmetry between the indexed song representation and the query representation is the prerequisite for cosine similarity to be a meaningful distance metric between them.

Engineering optimization. `_vocals_path()` checks for an existing `vocals.wav` at `separated/htdemucs/<song_stem>/vocals.wav` before invoking the separator. If found, the path is returned immediately, bypassing the 15–30 s inference cost. A singleton `_get_separator()` pattern ensures the Demucs model is loaded into memory only once per process.

5.2 Melody Extraction — Neural Pitch Tracking via BasicPitch

Model. BasicPitch (Spotify, ICASSP 2022; `src/audio/melody.py`) is a lightweight CNN that performs **simultaneous fundamental frequency (F0) estimation and note onset/offset detection** from raw audio waveforms. The model outputs three frame-level probability maps — onset, frame (sustain), and offset — which are post-processed into discrete MIDI note events.

Integration. `extract_melody()` calls `basic_pitch.inference.predict()` with the pretrained `ICASSP_2022_MODEL_PATH` checkpoint, returning a list of `(start_time, end_time, pitch, confidence, ...)` tuples. These are normalized into time-sorted `{pitch: int, start: float, end: float}` dictionaries — the canonical data contract between the extraction and cleaning stages.

Design rationale. BasicPitch is used as a **frozen feature extractor** with no fine-tuning. Known failure modes (octave errors, spurious short notes, vibrato-induced note splitting) are addressed by the downstream filtering stage rather than by retraining the model. This is the standard pattern for applying pretrained ML components in applied systems: compose with domain-specific heuristics rather than retrain.

5.3 Signal Cleaning — Domain-Specific Melody Filtering

Neural pitch trackers produce predictable artifact classes on real vocal audio: studio vocals exhibit breathiness, pitch drift, and vibrato; hummed input adds environmental noise and imprecise note onsets. `src/audio/melody_filter.py` implements a **five-stage deterministic cleaning pipeline** applied in strict sequence. Each stage targets one specific, known artifact class:

Stage	Function	Parameters	Artifact Targeted
1	<code>filter_invalid_notes</code>	MIDI [40–80], min duration 0.12 s	Sub-threshold pitches; sub-120 ms noise events unlikely to represent intentional vocal notes
2	<code>remove_overlapping_duplicates</code>	0.25 s window, keep max pitch	Multiple overlapping candidates per sung note — BasicPitch emits competing hypotheses that are resolved by non- maximum suppression
3	<code>remove_pitch_jumps</code>	Max sequential jump 12 semitones	Vocal melodies rarely span more than an octave between consecutive notes; larger jumps indicate detection errors
4	<code>remove_harmonic_notes</code>	Exact jumps of 12, 24, 36 semitones	Harmonic overtone misidentification — a well- documented failure mode in F0 trackers where upper harmonics are detected as the fundamental
5	<code>smooth_vibrato</code>	Adjacent pitch tolerance 2 semitones	Vibrato-induced note splitting — natural vocal vibrato (5–7 Hz oscillation) causes the tracker to segment a single sustained note into many short notes

The pipeline is **deliberately conservative**: each filter addresses a specific, known artifact class rather than applying generic smoothing. Thresholds were tuned empirically. The `clean_notes()` orchestrator applies all five stages, with stages composed as a simple sequential transformation of the note list.

5.4 Transposition-Invariant Representation — Interval Encoding

The transposition problem. Representing melody as absolute MIDI pitch numbers creates a **key-dependent representation**: the same melody performed in two different keys produces two different sequences, causing the similarity search to fail despite melodic identity. Since users hum in their

natural vocal range — almost never the song's original key — this is a critical failure mode that must be addressed at the representational level.

Solution. `src/audio/melody_normalizer.py` applies **first-order differencing** to the pitch sequence, converting absolute pitches `[p0, p1, ..., pn]` into a semitone interval sequence: `intervals[i] = pitch[i] - pitch[i-1]`.

Formal invariance property. Given a transposed pitch sequence `P' = P + k` (all pitches shifted by constant `k`):

$$P'[i] - P'[i-1] = (P[i] + k) - (P[i-1] + k) = P[i] - P[i-1]$$

The interval sequence is **identical regardless of `k`**. This single transformation is the mathematical foundation that enables cross-key matching between a user's hum and the indexed vocal melody.

Temporal normalization. The subsequent contour resampling step in the embedder (linear interpolation to 712 fixed points) additionally normalizes for tempo differences: two performances of the same melody at different tempos produce interval sequences of different lengths, which are mapped to the same fixed-length representation.

5.5 Feature Embedding — Deterministic 768-Dimensional Vector

`src/embedding/melody_embedder.py` converts the variable-length interval sequence into a **fixed-length, L2-normalized float32 vector** via three concatenated feature blocks. The design is analogous to a **bag-of-features** representation from classical information retrieval:

Block	Dimensions	Construction	Captures
Interval histogram	49	Normalized frequency of each semitone jump in <code>[-24, +24]</code>	Melodic <i>texture</i> — distribution of step sizes (conjunct stepwise motion vs. disjunct leaps)
Pitch-contour shape	712	Cumulative sum of intervals (reconstructed relative pitch trajectory), linearly resampled to 712 points, max-normalized	Melodic <i>shape</i> — the overall trajectory: rises, falls, arches, sequences
Summary statistics	7	Mean, std, min, max, count, positive-interval ratio, negative-interval ratio	Compact global descriptors of the interval distribution

Fixed-length rationale. Variable-length sequences cannot be directly compared via cosine similarity. Linear interpolation to 712 points maps sequences of any length into a common representational space. This trades precise temporal information for comparability — a deliberate design choice favoring retrieval correctness over temporal fidelity.

L2 normalization. The 768-dim `float32` vector is divided by its L2 norm, projecting it onto the unit hypersphere. This ensures cosine similarity reduces to a dot product: $\cos(A, B) = A \cdot B$ when $\|A\| = \|B\| = 1$, enabling efficient computation in Qdrant.

Deterministic vs. learned embeddings. The current embedding is fully hand-engineered — no training data, no neural network, no gradient descent. The trade-off is explicit:

Dimension	Deterministic (current)	Learned (future candidate)
Training data	None required	Labeled (hum, song) pairs required
Interpretability	Full — each dimension maps to a known feature	Limited — latent space
Adaptivity	Static; cannot model systematic hum deviations	Adaptive; can learn from user error patterns
Compute	O(n) NumPy operations	Neural network forward pass
Representational capacity	Bounded by feature design	Potentially unbounded with model scale

The architecture is explicitly designed to accommodate a learned embedding as a drop-in replacement for the current feature vector, making the path to higher accuracy a representation upgrade rather than a system rewrite.

5.6 Vector Retrieval — Approximate Nearest Neighbor Search via Qdrant

Storage. `src/database/qdrant_client.py` instantiates Qdrant in **local embedded mode** (`QdrantClient(path=...)`) — no external server process required. Each indexed song is stored as a `PointStruct` with a 768-dim vector and a JSON payload (`title`, `path`). The collection is configured with `Distance.COSINE`.

Indexing. Qdrant builds an **HNSW (Hierarchical Navigable Small World)** proximity graph over the vector space. HNSW enables **approximate nearest neighbor (ANN)** retrieval in $O(\log N)$ time, compared to the $O(N)$ cost of a brute-force linear scan. This means retrieval latency grows sub-

linearly with collection size, making the architecture scalable beyond the current proof-of-concept library.

Threshold filtering. `search_song()` filters Qdrant's raw results to cosine similarity ≥ 0.20 . Results below this threshold are discarded as low-confidence matches before returning to the caller. This threshold represents a precision-recall trade-off: too low admits false positives from noisy hums; too high rejects valid matches from imperfect input.

Re-indexing. `recreate_collection()` drops and recreates the Qdrant collection atomically, used when re-indexing after changes to the embedding pipeline or song library. `upload_song()` uses `client.upsert()` with sequential integer point IDs assigned by the indexing orchestrator.

6. Technology Stack

Technology	Role	Selection Rationale
Demucs 4.1.0 (<code>htdemucs</code>)	Vocal / source separation	State-of-the-art open-source BSS; <code>htdemucs</code> delivers the best separation quality among Demucs variants; GIL-releasing PyTorch inference
BasicPitch 0.4.0	Note event extraction	Pretrained, lightweight, open-source pitch detector (Spotify); avoids building a custom pitch tracker; GIL-releasing TensorFlow inference
Qdrant 1.18.0	Vector storage and ANN retrieval	Local embedded mode (no server dependency); native cosine similarity; HNSW index; clean Python client
PyTorch 2.13.0	Demucs inference runtime	Required runtime dependency of Demucs
TensorFlow 2.15.0	BasicPitch inference runtime	Required runtime dependency of BasicPitch
NumPy 1.26.4	Embedding math	Histogram, cumulative sum, linear interpolation, L2 normalization — all vectorized

Technology	Role	Selection Rationale
librosa 0.11.0 / soundfile 0.14.0 / sounddevice 0.5.5	Audio I/O and analysis	Standard, reliable Python audio stack
Streamlit 1.60.0	Web UI	Python-native; enables recording, upload, and search UI without a separate frontend stack
ThreadPoolExecutor (<code>concurrent.futures</code>)	Parallel indexing	Thread-level parallelism leveraging GIL release in PyTorch / TensorFlow; simpler than multiprocessing for this workload
pytest 9.1.1	Unit testing	Standard Python testing framework; covers all deterministic pipeline components

7. Engineering Challenges and Mitigations

Challenge	Root Cause	Mitigation Applied	Residual Gap
Humming inconsistency	Users hum at variable tempo, key, and onset precision	Interval encoding (key invariance) + contour resampling (tempo normalization)	Partial hums and mid-phrase starts produce truncated interval sequences with reduced matching accuracy
Recording noise	Environmental noise and microphone quality degrade F0 tracking accuracy	Five-stage filtering removes artifact classes post-extraction	No upstream signal enhancement (noise gate, spectral subtraction, voice activity detection) applied to query audio
Octave errors	F0 trackers misidentify harmonic overtones as the fundamental frequency	<code>remove_harmonic_notes</code> targets exact 12 / 24 / 36 semitone jumps	Near-octave errors (± 1 semitone from a harmonic interval) are not caught by the exact-match filter

Challenge	Root Cause	Mitigation Applied	Residual Gap
Non-learned embedding	Hand-engineered features cannot model systematic patterns in how users deviate from the original melody	Architectural limitation — no training data available	Learned embeddings (triplet loss, contrastive learning on hum/song pairs) would address this directly
Threshold generalization	Filter parameters tuned empirically on a small personal corpus	Tested against a personal song library	Parameters may not generalize to other libraries or humming styles; no formal optimization against labeled ground truth

8. Evaluation

8.1 Methodology

Evaluation was conducted informally on a personal song library (~15 songs). Query inputs were live microphone recordings of humming (20-second clips at 24 kHz, mono). No standardized publicly available QBH evaluation dataset (e.g., MIR-QBSH) was used.

8.2 Observed Results

Condition	Top-1 Accuracy	Primary Failure Mode
Clean recording (quiet room, in-tune hum, 15–20 s of distinctive melodic content)	~90% (18 / 20 attempts)	Partial hums; melodically ambiguous sections
Noisy recording (background noise, low-quality microphone, pitch instability)	Not formally measured	Pitch tracking degrades → noisy note events → embedding diverges from indexed representation

8.3 Scope and Limitations

These figures represent **practical usability on a personal library** under favorable conditions, not a validated benchmark. The sample size (20 attempts, ~15 songs) is insufficient for statistical significance. No claim is made that this accuracy generalizes to larger or more diverse libraries. A rigorous evaluation would require a standardized dataset, multiple test subjects, and statistical significance testing.

8.4 Unit Test Coverage

The `pytest` suite covers all deterministic pipeline components:

Test Module	Component Under Test	Coverage
<code>tests/test_melody_filter.py</code>	Five-stage filtering pipeline	Individual filter functions, edge cases, parameter boundaries, full pipeline end-to-end
<code>tests/test_melody_normalizer.py</code>	Interval conversion	Pitch-to-interval transformation correctness
<code>tests/test_melody_embedder.py</code>	Feature embedding	Vector dimensionality, L2 norm = 1, deterministic output
<code>tests/test_qdrant_client.py</code>	Vector database	Collection CRUD, search result structure, threshold filtering

9. Comparison with Existing Approaches

Approach	Query Modality	Key-Invariant	Tempo-Invariant	Hum-Compatible
Lyrics search	Typed text	N/A	N/A	No — requires lexical recall
Audio fingerprinting (Shazam)	Studio recording audio	No	No	No — hash is recording-specific

Approach	Query Modality	Key-Invariant	Tempo-Invariant	Hum-Compatible
Collaborative filtering	Listening history	N/A	N/A	No — solves recommendation, not recognition
Hum-to-Search (this project)	Hummed / uploaded audio	Yes (interval encoding)	Yes (contour resampling)	Yes — designed for hum input

Key distinction. Audio fingerprinting and lyrics search solve the **known-item identification** problem when the user has the original recording or the words. Hum-to-Search solves the same problem when the user has *only the melody in memory* — a strictly harder input modality that conventional systems cannot address by construction.

10. Future Roadmap

Improvement	Category	Expected Impact
Learned melody embeddings — triplet loss or contrastive learning on labeled (hum, song) pairs	Representation	Adaptive to systematic humming deviations; higher capacity than hand-engineered features
Transformer-based sequence models — pitch / interval tokens or spectrogram frames as input	Representation	Captures long-range melodic dependencies lost in fixed-length histogram encoding
Standardized evaluation — MIR-QBSH or a custom multi-user hum corpus with ground-truth labels	Evaluation	Rigorous accuracy benchmarking and statistical significance testing
Query-side signal enhancement — noise gate, spectral subtraction, voice activity detection	Preprocessing	Improves F0 tracking accuracy on noisy recordings before pitch extraction
Humming data augmentation — pitch-shifting, tempo-warping, noise injection	Training	Simulates diverse humming styles for robustness and model training
	Retrieval	

Improvement	Category	Expected Impact
Deep metric learning — learned similarity space optimized for hum-to-song distance		Distance metric optimized specifically for this matching task vs. generic cosine distance
Mobile / edge deployment — ONNX export, quantized inference	Deployment	Removes dependency on desktop PyTorch / TensorFlow; enables on-device real-time search

11. Engineering Learnings

Composable single-responsibility architecture. Structuring the system as strictly single-responsibility stages — separation → extraction → cleaning → normalization → embedding → search — with explicit, typed data contracts between stages made each component independently testable and debuggable. Issues were isolated to the stage that produced them rather than requiring end-to-end debugging.

Invariance engineering over model training. The core design insight was identifying *what varies* between two performances of the same melody (key, tempo, noise, vocal quality) and encoding mathematical invariances directly into the feature representation. Interval differencing for key invariance and contour resampling for tempo normalization both eliminate variability at the representational level — a more robust approach than relying on a model to learn these invariances implicitly from data.

Pretrained model integration with explicit downstream compensation. Using Demucs and BasicPitch as frozen feature extractors avoided reinventing BSS and pitch detection from scratch. This required identifying each model's known failure modes (octave errors, spurious short notes, vibrato splitting) and building explicit downstream compensation logic — a standard pattern in applied ML: compose pretrained models with domain-specific heuristics rather than retrain.

Vector retrieval as a general pattern. The "encode → store in vector database → retrieve by similarity" pattern generalizes from text and image search to any domain where entities can be embedded into a metric space. This project demonstrates the pattern on engineered audio features, reinforcing its domain-agnostic applicability.

Production engineering decisions. Disk caching of expensive intermediate artifacts (vocal stems), thread-pool parallelism leveraging GIL release in PyTorch / TensorFlow, and a UI with real-time progress feedback are engineering decisions that distinguish a deployable system from a research notebook. Each decision addresses a specific operational constraint: latency, throughput, and user experience respectively.

12. Conclusion

Hum-to-Search is a complete, end-to-end AI system for query-by-humming song recognition, built from composable, single-responsibility components: Demucs for blind source separation, BasicPitch for neural pitch tracking, a custom five-stage signal cleaning pipeline, first-order interval differencing for transposition invariance, a deterministic 768-dimensional feature embedding, and Qdrant's HNSW index for approximate nearest neighbor retrieval.

The system solves a problem that conventional approaches cannot address by construction: identifying a song from a hummed melody alone, with invariance to the key and tempo of the performance. Demonstrated accuracy of ~90% on clean recordings over a personal library establishes the viability of the architecture without requiring any custom-trained models — every component is either a frozen pretrained model or a deterministic, hand-engineered algorithm.

The primary limitation of the current system is that a non-learned embedding cannot adapt to systematic patterns in how users deviate from the original melody. The architectural foundation — asymmetric pipelines, interval encoding, vector retrieval — is explicitly designed to support learned embeddings as a drop-in replacement, making the path to improved accuracy a representation upgrade rather than a system rewrite.

This report describes features implemented in the repository at the time of writing. Capabilities listed under Section 10 are future work and are not implemented in the current codebase.